

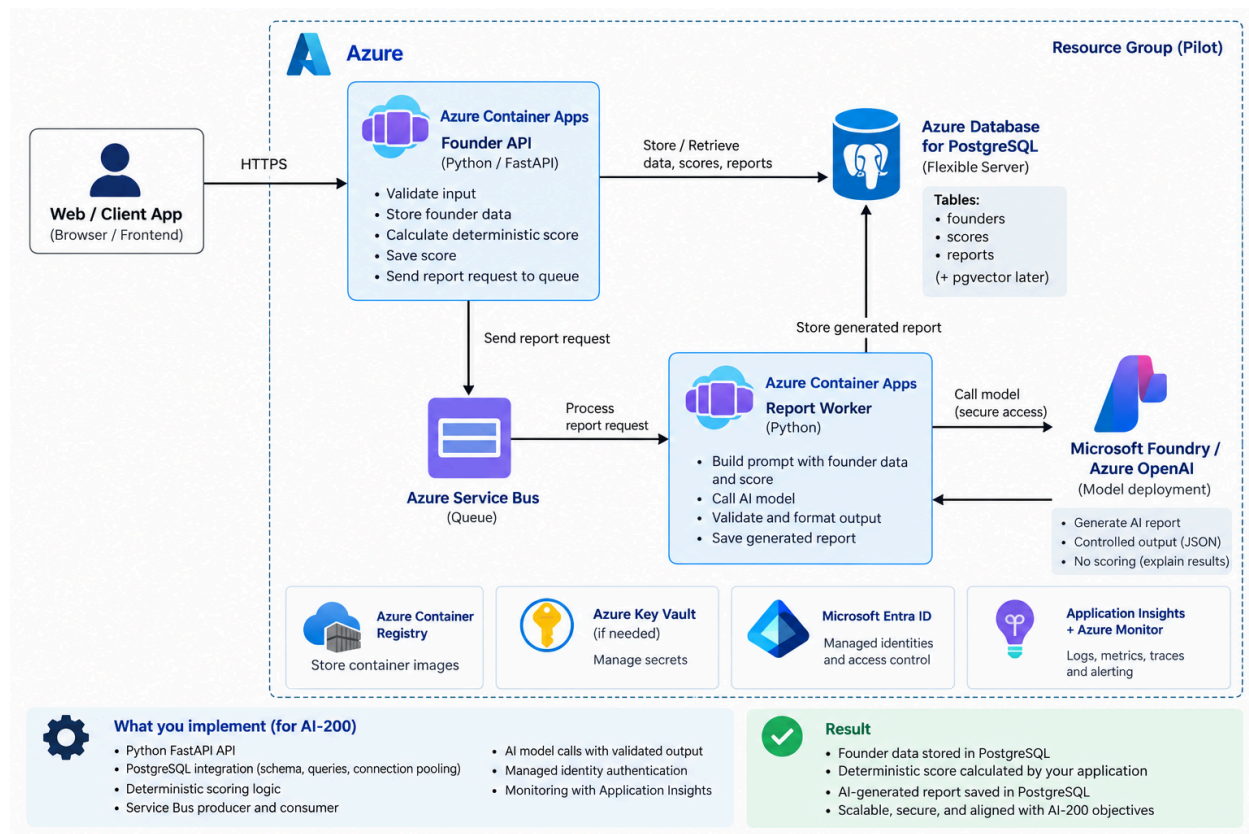
AI-200 Q&A (October 2026)

Q (P.J.)

“I’m building an application that accepts founder data, stores it in PostgreSQL, calculates a deterministic score, and uses AI to generate a report. What Azure architecture would you recommend for a small pilot, and which parts should I implement myself to prepare for AI-200?”

Answer:

For a small pilot, I would use Azure Container Apps for the application, Azure Database for PostgreSQL Flexible Server to store founder data, scores, and reports, and Microsoft Foundry/Azure OpenAI to generate the AI-based report. The deterministic score should be calculated in your own application code rather than by the AI model. To prepare for AI-200, I would implement the Python API, PostgreSQL integration, scoring logic, managed identity authentication, Service Bus messaging for asynchronous report generation, AI model calls, and Application Insights monitoring yourself.



Q: (G.B.)

Most proba



Gursharan B 10:45 AM

Can you briefly explain about "session" consistency level in CosmosDB? How does replica writes behave in this?

Answer:

In Azure Cosmos DB, Session consistency means a client session gets read-your-own-writes behavior. After your application writes or updates an item, subsequent reads from that same session will see that write, even if replication to all regions has not fully completed.

For replica writes, Cosmos DB still replicates the write to other replicas/regions asynchronously, depending on the account configuration. Session consistency does not require every replica to be updated before acknowledging the write. Instead, Cosmos DB uses a session token to track the latest version seen by that client. On subsequent reads, that token helps ensure the request is served from a replica that has caught up to at least that version.

So, in short: writes replicate normally, potentially with some lag, but the same client session will not read an older version than the one it has already written or observed.

<end of document>